

LC&CF [ASSIGNMENT]

UDAY CHIRRA
2303A510G2

1. Delete and Earn

```
class Solution {
    public int deleteAndEarn(int[] nums) {
        int max = 0;
        for (int num : nums) {
            max = Math.max(max, num);
        }
        int[] points = new int[max + 1];
        for (int num : nums) {
            points[num] += num;
        }
        int prev2 = 0;
        int prev1 = 0;
        for (int i = 1; i <= max; i++) {
            int current = Math.max(prev1, prev2 + points[i]);
            prev2 = prev1;
            prev1 = current;
        }
        return prev1;
    }
}
```

2. Partition Equal Subset Sum

```
class Solution {
    public boolean canPartition(int[] nums) {
        int total = 0;
        for (int num : nums) {
            total += num;
        }
        if (total % 2 != 0) {
            return false;
        }
        int target = total / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int num : nums) {
            for (int j = target; j >= num; j--) {
```

```

        dp[j] = dp[j] || dp[j - num];
    }
}
return dp[target];
}
}

```

3. Target Sum

```

class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int sum = 0;
        for (int num : nums) {
            sum += num;
        }
        if (Math.abs(target) > sum || (sum + target) % 2 != 0) {
            return 0;
        }
        int required = (sum + target) / 2;
        int[] dp = new int[required + 1];
        dp[0] = 1;
        for (int num : nums) {
            for (int j = required; j >= num; j--) {
                dp[j] += dp[j - num];
            }
        }
        return dp[required];
    }
}

```

4. Minimum Cost For Tickets

```

class Solution {
    public int mincostTickets(int[] days, int[] costs) {
        int n = days.length;
        int[] dp = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            dp[i] = dp[i - 1] + costs[0];
            int j = i - 1;
            while (j >= 0 && days[j] >= days[i - 1] - 6) {
                j--;
            }
            dp[i] = Math.min(dp[i], dp[j + 1] + costs[1]);
            j = i - 1;
            while (j >= 0 && days[j] >= days[i - 1] - 29) {
                j--;
            }
        }
    }
}

```

```

    }
    dp[i] = Math.min(dp[i], dp[j + 1] + costs[2]);
}
return dp[n];
}
}

```

5. Last Stone Weight II

```

class Solution {
    public int lastStoneWeightII(int[] stones) {
        int sum = 0;
        for (int stone : stones) {
            sum += stone;
        }
        int target = sum / 2;
        int[] dp = new int[target + 1];
        for (int stone : stones) {
            for (int j = target; j >= stone; j--) {
                dp[j] = Math.max(dp[j], dp[j - stone] + stone);
            }
        }
        return sum - 2 * dp[target];
    }
}

```

6. Codeforces 455A - Boredom

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long[] freq = new long[100001];
        for (int i = 0; i < n; i++) {
            freq[sc.nextInt()]++;
        }
        long prev2 = 0;
        long prev1 = 0;
        for (int i = 1; i <= 100000; i++) {
            long current = Math.max(prev1, prev2 + freq[i] * i);
            prev2 = prev1;
            prev1 = current;
        }
        System.out.println(prev1);
    }
}

```

```
}
```

7. Codeforces 1195C - Basketball Exercise

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long[] top = new long[n];
        long[] bottom = new long[n];
        for (int i = 0; i < n; i++) {
            top[i] = sc.nextLong();
        }
        for (int i = 0; i < n; i++) {
            bottom[i] = sc.nextLong();
        }
        long topPrev2 = 0;
        long topPrev1 = 0;
        long bottomPrev2 = 0;
        long bottomPrev1 = 0;
        for (int i = 0; i < n; i++) {
            long newTop = Math.max(topPrev1, bottomPrev2 + top[i]);
            long newBottom = Math.max(bottomPrev1, topPrev2 + bottom[i]);
            topPrev2 = topPrev1;
            topPrev1 = newTop;
            bottomPrev2 = bottomPrev1;
            bottomPrev1 = newBottom;
        }
        System.out.println(Math.max(topPrev1, bottomPrev1));
    }
}
```

8. Codeforces 1097B - Petr and a Combination Lock

```
import java.util.*;
public class Main {
    static int n;
    static int[] a;
    static boolean dfs(int index, int sum) {
        if (index == n) {
            return sum % 360 == 0;
        }
        return dfs(index + 1, sum + a[index]) ||
            dfs(index + 1, sum - a[index]);
    }
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    n = sc.nextInt();  
    a = new int[n];  
    for (int i = 0; i < n; i++) {  
        a[i] = sc.nextInt();  
    }  
    System.out.println(dfs(0, 0) ? "YES" : "NO");  
}  
}
```